Tuning the Two-Stage Pipeline: Top-K Ratio & P@1, Worked Out by Hand

Why the re-ranker can only fix what the retriever already found — the "retrieve-K $\geq 4\times$ rerank-K" rule of thumb — shown through a worked case where a too-small retrieval window caps final precision no matter how good the re-ranker.

What this document does. It defines the two knobs of a retrieve-then- rerank pipeline (K_{ret} and K_{rank}), shows with Precision@1 and recall how a stingy first stage hard-caps the final answer quality, derives the common "4×" heuristic, and explains the latency cost that stops you from just setting K_{ret} huge. Numbers from Appendix A.

Prerequisite: bi- vs cross-encoder (#22) — this PDF sizes the two stages that architecture sets up.

1 The two knobs

A two-stage pipeline has two window sizes:

- K_{ret} — how many candidates the cheap bi-encoder *retrieves*.
- K_{rank} — how many of those the expensive cross-encoder *re-ranks* and ultimately returns ($K_{\text{rank}} \leq K_{\text{ret}}$).

The cross-encoder re-orders; it cannot *add*. If a relevant document is not in the retrieved K_{ret}, no amount of re-ranking can surface it.

> **Intuition**
>
> The re-ranker is a brilliant editor working only from the pile of articles the researcher handed them. If the researcher (retriever) never found the key article, the editor cannot put it on the front page — it is not in the pile. So the first stage sets a *ceiling* on quality (it determines what is even *available*), and the second stage decides how close to that ceiling you get (it determines the *ordering*). Tune the retriever for recall; tune the re-ranker for precision.

2 The ceiling, made concrete

Suppose for a query the truly-best document sits at rank 12 in the bi-encoder's ordering (retrieval is decent but imperfect — the best answer is a bit buried).

Case A — $K_{\text{ret}} = 10$. The best document (rank 12) is *not retrieved*. The cross-encoder re-ranks documents 1–10, none of which is the true best. Final Precision@1 for this query: 0 — and *nothing* the re-ranker does can change it. The miss happened upstream.

Case B — $K_{\text{ret}} = 50$. Now rank 12 *is* retrieved. The cross-encoder reads all 50 jointly, recognises it as the best answer, and lifts it to rank 1. Final Precision@1: 1. Same re-ranker, same query — only the retrieval window changed.

	K_{ret}	best doc retrieved?	final P@1
Case A	10	no (it was rank 12)	0
Case B	50	yes	1

Mini-example · "P@1 didn't improve" is usually a recall bug

A common symptom: you add a great cross-encoder and Precision@1 barely moves. The instinct is to blame the re-ranker — almost always wrong. If the right answers are not in K_{ret}, the re-ranker is re-sorting a pile that lacks them, so its ceiling is low. The fix is not a better re-ranker; it is a *bigger retrieval window* (raise K_{ret}) or a better first-stage retriever. Measure retrieval recall@K_{ret} (#12) first — if it is low, that is your bug.

3 The 4× rule of thumb

You want K_{ret} large enough that retrieval recall is near 1 — i.e. the relevant documents are almost surely *in the pile* — before the re-ranker does its job. Empirically, retrieval recall keeps climbing well past K_{rank}, and a widely used heuristic is

$$\boxed{K_{\text{ret}} \gtrsim 4 \times K_{\text{rank}}}$$

So to return a re-ranked top-10, retrieve ∼40–50; for a top-5, retrieve ∼20. The ratio buys recall headroom so the re-ranker is choosing from a pile that actually contains the answers. Many teams land on $K_{\text{ret}} \approx 50$ feeding a K_{rank} of 10.

Intuition

Why not just set $K_{\text{ret}} = 1000$ and be safe? Because the cross-encoder runs *once per retrieved candidate* (#22), so re-ranking cost grows linearly with K_{ret} if you re-rank them all — and latency is a hard product constraint. The 4× rule is the sweet spot: wide enough that recall is near-saturated (extra candidates would add almost nothing, per the diminishing-returns curve of #17), narrow enough that the re-ranker stays within budget. Past the recall knee, more candidates buy latency, not answers.

4 Putting the knobs together

- **Size K_{ret} by recall:** raise it until retrieval recall@K_{ret} clears your target (e.g. 0.95). That sets the ceiling.
- **Size K_{rank} by latency:** re-rank as many as your cross-encoder budget allows, typically $K_{\text{ret}}/4$ or so.
- **Apply access controls at the right place:** filter by permissions/ACLs *during or before* retrieval, not after re-ranking — otherwise you spend cross-encoder compute scoring documents the user may never see, and risk leaking their existence.
- **Diagnose top-down:** bad P@1 ⇒ check retrieval recall first (#12), then the re-ranker. Fixing the wrong stage wastes effort.

Equation cheat-sheet

$$K_{\text{rank}} \le K_{\text{ret}}, \qquad K_{\text{ret}} \gtrsim 4\,K_{\text{rank}}, \qquad \text{P@1} = \frac{\text{best doc at rank 1?}}{1}$$

retriever sets the *ceiling* (recall); re-ranker reaches it (precision); flat P@1 ⇒ raise K_{ret}

Appendix A · The ceiling effect

```
best_doc_rank = 12          # where the retriever ranked the true best answer

def final_p_at_1(K_ret):
    # cross-encoder can only pick from the retrieved pile
    return 1 if best_doc_rank <= K_ret else 0

final_p_at_1(10)   # 0  -> best answer (rank 12) never retrieved; re-ranker helpless
final_p_at_1(50)   # 1  -> retrieved, then lifted to #1 by the cross-encoder

# rule of thumb:  K_ret >= 4 * K_rank   (e.g. retrieve 40-50 to return a top-10)
```

Internal learning material. Illustrative of the ceiling effect. v1.0